

Unprotected Admin Functionality

Name: Matyas Abraham

ID: CTC-207-26

Lab URL:

portswigger.net/web-security/access-control/lab-unprotected-admin-functionality

Reconnaissance / Enumeration

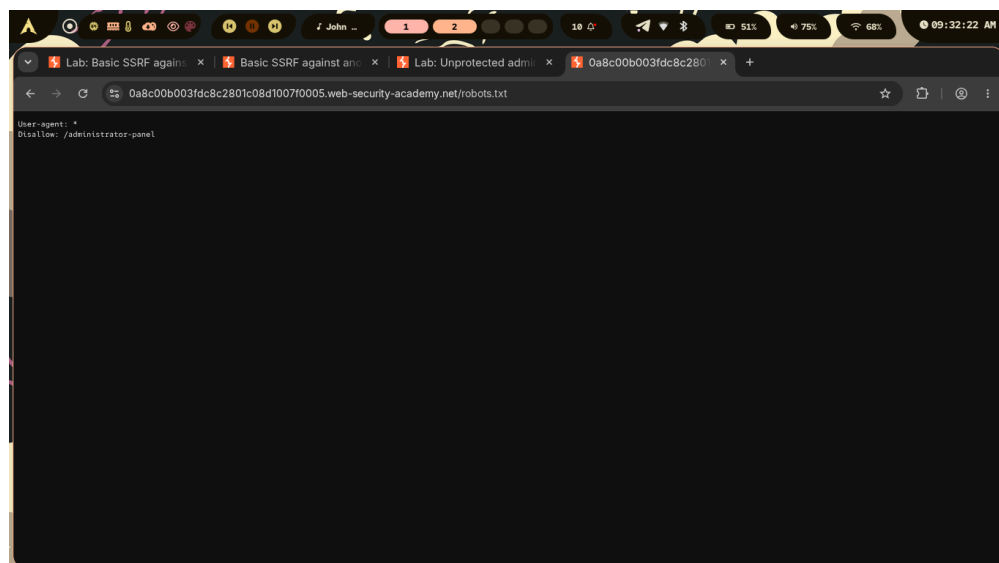
The site itself doesn't link to any kind of admin area anywhere in its normal navigation, so the first step was hidden-content discovery. I ran gobuster against the site root using a directory wordlist to see what else might be sitting on the server that isn't linked from the UI:

```
gobuster dir -u https://<lab-id>.web-security-academy.net/ -w words.txt
```



```
gobuster dir -u https://0a8c00b003fdc8c2801c08d1007f0005.web-security-academy.net/ -w words.txt
=====
Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
=====
[+] Url:             https://0a8c00b003fdc8c2801c08d1007f0005.web-security-academy.net/
[+] Method:          GET
[+] Threads:         10
[+] Wordlist:         words.txt
[+] Negative Status codes: 404
[+] User Agent:      gobuster/3.8.2
[+] Timeout:         10s
=====
Starting gobuster in directory enumeration mode
=====
robots.txt           (Status: 200) [Size: 45]
Progress: 3 / 3 (100.00%)
=====
Finished
=====
```

The scan turned up robots.txt, which is worth checking on basically every engagement - it's meant to tell search engines what not to crawl, but that list of "don't look here" paths is often the most interesting part of the site. Reading it directly gave away the exact path to the admin area:

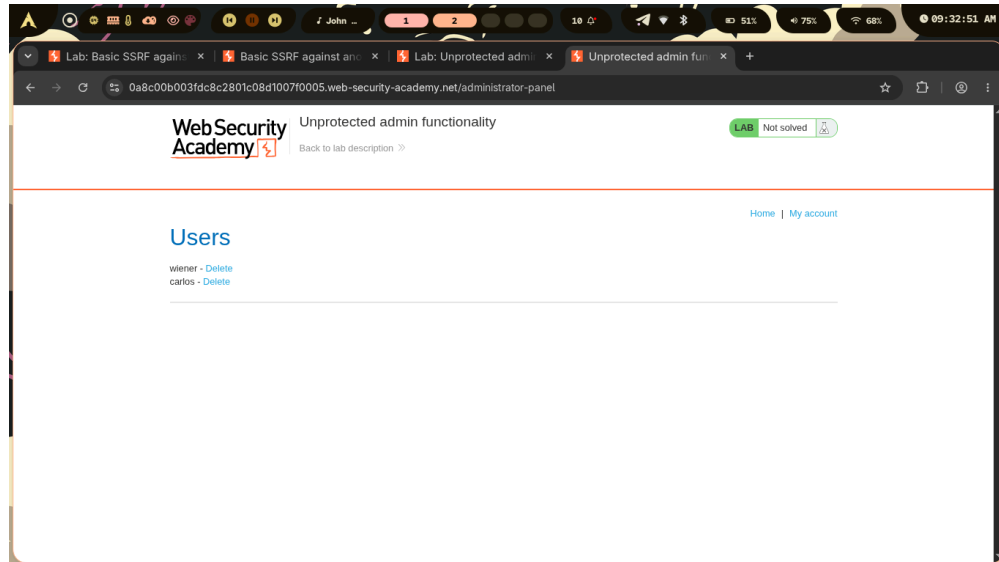


```
User-agent: *
Disallow: /administrator-panel
```

Disallow: /administrator-panel

Vulnerability Discovered

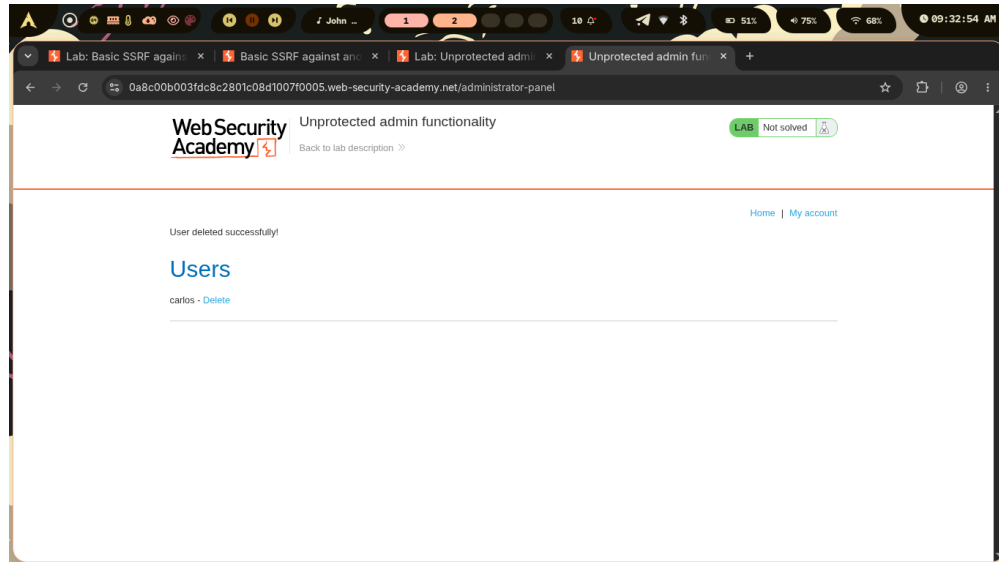
Browsing straight to /administrator-panel loaded the admin interface immediately - no login prompt, no session check, nothing. It listed every user on the system (wiener and carlos) with a Delete link sitting right next to each one.



This is broken access control (CWE-284 / OWASP A01): sensitive admin functionality was reachable by anyone who simply knew (or discovered) the URL, with no authentication or authorization check gating it at all. The only thing standing between a normal visitor and full admin actions was the fact that the path wasn't linked anywhere - which robots.txt undid for free.

Exposed Information & Security Impact

With the panel wide open, I first used the Delete link next to wiener just to confirm there was genuinely no authorization check at all - and there wasn't, the deletion went through instantly with no prompt or confirmation step.



I then used the same unprotected Delete link to remove carlos as well, which is the specific account this lab requires deleting to mark it as solved.

The impact here isn't really about information disclosure - it's about unauthorized action. Anyone who finds this URL (and, as shown, that's trivial - a two-second robots.txt check) can delete arbitrary user accounts, or whatever other admin actions this panel exposes, without ever proving they're an admin or even logging in. On a real system this could mean account deletion, privilege changes, or data manipulation carried out by a completely anonymous, unauthenticated visitor.

Remediation Recommendations

1. Enforce server-side authentication and authorization checks on every admin route, not just ones that happen to be linked from a visible admin login page. "Security by obscurity" - assuming a URL is safe because it isn't linked anywhere - isn't real access control.
2. Never list sensitive paths in robots.txt. If a directory genuinely needs to be excluded from search engines, protect it with authentication first; robots.txt is a courtesy for well-behaved crawlers, not a security boundary, and it's public and readable by anyone.
3. Apply the principle of least privilege and role-based access control - admin-only actions like deleting a user should verify the requester actually holds an admin role/session server-side on every request, rather than trusting that only admins will know or guess the URL.

Proof of Solve

Screenshot below shows the lab marked as Solved:

